JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class InputStream

java.lang.Object
    java.io.InputStream

All Implemented Interfaces:

Closeable, AutoCloseable

Direct Known Subclasses:

AudioInputStream, ByteArrayInputStream, FileInputStream, FilterInputStream, ObjectInputStream, PipedInputStream, SequenceInputStream, StringBufferInputStream

---

```
public abstract class InputStream
extends Object
implements Closeable
```

This abstract class is the superclass of all classes representing an input stream of bytes.

Applications that need to define a subclass of `InputStream` must always provide a method that returns the next byte of input.

Since:

1.0

See Also:

BufferedInputStream, ByteArrayInputStream, DataInputStream, FilterInputStream, read(), OutputStream, PushbackInputStream

Constructor Summary

Constructors

Constructor	Description
InputStream()	Constructor for subclasses to call.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
int	available()	Returns an estimate of the number of bytes that can be read (or skipped over) from this input

		stream without blocking, which may be 0, or 0 when end of stream is detected.
void	**close**()	Closes this input stream and releases any system resources associated with the stream.
void	**mark**(int readlimit)	Marks the current position in this input stream.
boolean	**markSupported**()	Tests if this input stream supports the `mark` and `reset` methods.
static InputStream	**nullInputStream**()	Returns a new `InputStream` that reads no bytes.
abstract int	**read**()	Reads the next byte of data from the input stream.
int	**read**(byte[] b)	Reads some number of bytes from the input stream and stores them into the buffer array `b`.
int	**read**(byte[] b, int off, int len)	Reads up to `len` bytes of data from the input stream into an array of bytes.
byte[]	**readAllBytes**()	Reads all remaining bytes from the input stream.
int	**readNBytes**(byte[] b, int off, int len)	Reads the requested number of bytes from

		the input stream into the given byte array.
byte[]	**readNBytes**(int len)	Reads up to a specified number of bytes from the input stream.
void	**reset**()	Repositions this stream to the position at the time the `mark` method was last called on this input stream.
long	**skip**(long n)	Skips over and discards `n` bytes of data from this input stream.
void	**skipNBytes**(long n)	Skips over and discards exactly `n` bytes of data from this input stream.
long	**transferTo**(OutputStream out)	Reads all bytes from this input stream and writes the bytes to the given output stream in the order that they are read.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

InputStream

```
public InputStream()
```

Constructor for subclasses to call.

Method Details

nullInputStream

```
public static InputStream nullInputStream()
```

Returns a new `InputStream` that reads no bytes. The returned stream is initially open. The stream is closed by calling the `close()` method. Subsequent calls to `close()` have no effect.

While the stream is open, the `available()`, `read()`, `read(byte[])`, `read(byte[], int, int)`, `readAllBytes()`, `readNBytes(byte[], int, int)`, `readNBytes(int)`, `skip(long)`, `skipNBytes(long)`, and `transferTo()` methods all behave as if end of stream has been reached. After the stream has been closed, these methods all throw `IOException`.

The `markSupported()` method returns `false`. The `mark()` method does nothing, and the `reset()` method throws `IOException`.

Returns:

an `InputStream` which contains no bytes

Since:

11

read

```
public abstract int read()
                  throws IOException
```

Reads the next byte of data from the input stream. The value byte is returned as an `int` in the range `0` to `255`. If no byte is available because the end of the stream has been reached, the value `-1` is returned. This method blocks until input data is available, the end of the stream is detected, or an exception is thrown.

Returns:

the next byte of data, or `-1` if the end of the stream is reached.

Throws:

`IOException` - if an I/O error occurs.

read

```
public int read(byte[] b)
         throws IOException
```

Reads some number of bytes from the input stream and stores them into the buffer array `b`. The number of bytes actually read is returned as an integer.

This method blocks until input data is available, end of file is detected, or an exception is thrown.

If the length of `b` is zero, then no bytes are read and `0` is returned; otherwise, there is an attempt to read at least one byte. If no byte is available because the stream is at the end of the file, the value `-1` is returned; otherwise, at least one byte is read and stored into `b`.

The first byte read is stored into element `b[0]`, the next one into `b[1]`, and so on. The number of bytes read is, at most, equal to the length of `b`. Let *k* be the number of bytes actually read; these bytes will be stored in elements `b[0]` through `b[`*k*`-1]`, leaving elements `b[`*k*`]` through `b[b.length-1]` unaffected.

Implementation Requirements:

The `read(b)` method for class `InputStream` has the same effect as:

```
read(b, 0, b.length)
```

Parameters:

`b` - the buffer into which the data is read.

Returns:

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the stream has been reached.

Throws:

`IOException` - If the first byte cannot be read for any reason other than the end of the file, if the input stream has been closed, or if some other I/O error occurs.

`NullPointerException` - if `b` is `null`.

See Also:

`read(byte[], int, int)`

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads up to `len` bytes of data from the input stream into an array of bytes. An attempt is made to read as many as `len` bytes, but a smaller number may be read. The number of bytes actually read is returned as an integer.

This method blocks until input data is available, end of file is detected, or an exception is thrown.

If `len` is zero, then no bytes are read and `0` is returned; otherwise, there is an attempt to read at least one byte. If no byte is available because the stream is at end of file, the value `-1` is returned; otherwise, at least one byte is read and stored into `b`.

The first byte read is stored into element `b[off]`, the next one into `b[off+1]`, and so on. The number of bytes read is, at most, equal to `len`. Let *k* be the number of bytes actually read; these bytes will be stored in elements `b[off]` through `b[off+`*k*`-1]`, leaving elements `b[off+`*k*`]` through `b[off+len-1]` unaffected.

In every case, elements `b[0]` through `b[off-1]` and elements `b[off+len]` through `b[b.length-1]` are unaffected.

Implementation Requirements:

The `read(b, off, len)` method for class `InputStream` simply calls the method `read()` repeatedly. If the first such call results in an `IOException`, that exception is returned from the call to the `read(b, off, len)` method. If any subsequent call to `read()` results in a `IOException`, the exception is caught and treated as if it were end of file; the bytes read up to that point are stored into `b` and the number of bytes read before the exception occurred is returned. The default implementation of this method blocks until the requested amount of input data `len` has been read, end of file is detected, or an exception is thrown. Subclasses are encouraged to provide a more efficient implementation of this method.

Parameters:

`b` - the buffer into which the data is read.

`off` - the start offset in array `b` at which the data is written.

`len` - the maximum number of bytes to read.

Returns:

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the stream has been reached.

Throws:

`IOException` - If the first byte cannot be read for any reason other than end of file, or if the input stream has been closed, or if some other I/O error occurs.

`NullPointerException` - If `b` is `null`.

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

See Also:

`read()`

readAllBytes

```
public byte[] readAllBytes()
                     throws IOException
```

Reads all remaining bytes from the input stream. This method blocks until all remaining bytes have been read and end of stream is detected, or an exception is thrown. This method does not close the input stream.

When this stream reaches end of stream, further invocations of this method will return an empty byte array.

Note that this method is intended for simple cases where it is convenient to read all bytes into a byte array. It is not intended for reading input streams with large amounts of data.

The behavior for the case where the input stream is *asynchronously closed*, or the thread interrupted during the read, is highly input stream specific, and therefore not specified.

If an I/O error occurs reading from the input stream, then it may do so after some, but not all, bytes have been read. Consequently the input stream may not be at end of stream and may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

Implementation Requirements:

This method invokes `readNBytes(int)` with a length of `Integer.MAX_VALUE`.

Returns:

a byte array containing the bytes read from this input stream

Throws:

`IOException` - if an I/O error occurs

`OutOfMemoryError` - if an array of the required size cannot be allocated.

Since:

9

readNBytes

```
public byte[] readNBytes(int len)
                  throws IOException
```

Reads up to a specified number of bytes from the input stream. This method blocks until the requested number of bytes has been read, end of stream is detected, or an exception is thrown. This method does not close the input stream.

The length of the returned array equals the number of bytes read from the stream. If `len` is zero, then no bytes are read and an empty byte array is returned. Otherwise, up to `len` bytes are read from the stream. Fewer than `len` bytes may be read if end of stream is encountered.

When this stream reaches end of stream, further invocations of this method will return an empty byte array.

Note that this method is intended for simple cases where it is convenient to read the specified number of bytes into a byte array. The total amount of memory allocated by this method is proportional to the number of bytes read from the stream which is bounded by `len`. Therefore, the method may be safely called with very large values of `len` provided sufficient memory is available.

The behavior for the case where the input stream is *asynchronously closed*, or the thread interrupted during the read, is highly input stream specific, and therefore not specified.

If an I/O error occurs reading from the input stream, then it may do so after some, but not all, bytes have been read. Consequently the input stream may not be at end of stream and may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

Implementation Note:

The number of bytes allocated to read data from this stream and return the result is bounded by `2*(long)len`, inclusive.

Parameters:

`len` - the maximum number of bytes to read

Returns:

a byte array containing the bytes read from this input stream

Throws:

`IllegalArgumentException` - if `length` is negative

`IOException` - if an I/O error occurs

`OutOfMemoryError` - if an array of the required size cannot be allocated.

Since:

11

readNBytes

```
public int readNBytes(byte[] b,
                      int off,
                      int len)
               throws IOException
```

Reads the requested number of bytes from the input stream into the given byte array. This method blocks until `len` bytes of input data have been read, end of stream is detected, or an exception is thrown. The number of bytes actually read, possibly zero, is returned. This method does not close the input stream.

In the case where end of stream is reached before `len` bytes have been read, then the actual number of bytes read will be returned. When this stream reaches end of stream, further invocations of this method will return zero.

If `len` is zero, then no bytes are read and `0` is returned; otherwise, there is an attempt to read up to `len` bytes.

The first byte read is stored into element `b[off]`, the next one in to `b[off+1]`, and so on. The number of bytes read is, at most, equal to `len`. Let *k* be the number of bytes actually read; these bytes will be stored in elements `b[off]` through `b[off+`*k*`-1]`, leaving elements `b[off+`*k*` ]` through `b[off+len-1]` unaffected.

The behavior for the case where the input stream is *asynchronously closed*, or the thread interrupted during the read, is highly input stream specific, and therefore not specified.

If an I/O error occurs reading from the input stream, then it may do so after some, but not all, bytes of `b` have been updated with data from the input stream. Consequently the input stream and `b` may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

Parameters:

`b` - the byte array into which the data is read

`off` - the start offset in `b` at which the data is written

`len` - the maximum number of bytes to read

Returns:

the actual number of bytes read into the buffer

Throws:

`IOException` - if an I/O error occurs

`NullPointerException` - if `b` is `null`

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

Since:

9

skip

```
public long skip(long n)
          throws IOException
```

Skips over and discards `n` bytes of data from this input stream. The `skip` method may, for a variety of reasons, end up skipping over some smaller number of bytes, possibly `0`. This may result from any of a number of conditions; reaching end of file before `n` bytes have been skipped is only one possibility. The actual number of bytes skipped is returned. If `n` is negative, the `skip` method for class `InputStream` always returns 0, and no bytes are skipped. Subclasses may handle the negative value differently.

Implementation Requirements:

The `skip` method implementation of this class creates a byte array and then repeatedly reads into it until `n` bytes have been read or the end of the stream has been reached. Subclasses are encouraged to provide a more efficient implementation of this method. For instance, the implementation may depend on the ability to seek.

Parameters:

`n` - the number of bytes to be skipped.

Returns:

the actual number of bytes skipped which might be zero.

Throws:

`IOException` - if an I/O error occurs.

See Also:

`skipNBytes(long)`

skipNBytes

```
public void skipNBytes(long n)
                throws IOException
```

Skips over and discards exactly `n` bytes of data from this input stream. If `n` is zero, then no bytes are skipped. If `n` is negative, then no bytes are skipped. Subclasses may handle the negative value differently.

This method blocks until the requested number of bytes has been skipped, end of file is reached, or an exception is thrown.

If end of stream is reached before the stream is at the desired position, then an `EOFException` is thrown.

If an I/O error occurs, then the input stream may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

Implementation Requirements:

If `n` is zero or negative, then no bytes are skipped. If `n` is positive, the default implementation of this method invokes `skip()` repeatedly with its parameter equal to the remaining number of bytes to skip until the requested number of bytes has been skipped or an error condition occurs. If at any point the return value of `skip()` is negative or greater than the remaining number of bytes to be skipped, then an `IOException` is thrown. If `skip()` ever returns zero, then `read()` is invoked to read a single byte, and if it returns `-1`, then an `EOFException` is thrown. Any exception thrown by `skip()` or `read()` will be propagated.

Implementation Note:

Subclasses are encouraged to provide a more efficient implementation of this method.

Parameters:

`n` - the number of bytes to be skipped.

Throws:

`EOFException` - if end of stream is encountered before the stream can be positioned `n` bytes beyond its position when this method was invoked.

`IOException` - if the stream cannot be positioned properly or if an I/O error occurs.

Since:

12

See Also:

`skip(long)`

available

```
public int available()
             throws IOException
```

Returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking, which may be 0, or 0 when end of stream is detected. The read might be on the same thread or another thread. A single read or skip of this many bytes will not block, but may read or skip fewer bytes.

Note that while some implementations of `InputStream` will return the total number of bytes in the stream, many will not. It is never correct to use the

return value of this method to allocate a buffer intended to hold all data in this stream.

A subclass's implementation of this method may choose to throw an `IOException` if this input stream has been closed by invoking the `close()` method.

API Note:

This method should be overridden by subclasses.

Implementation Requirements:

The `available` method of `InputStream` always returns `0`.

Returns:

an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking or `0` when it reaches the end of the input stream.

Throws:

`IOException` - if an I/O error occurs.

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Implementation Requirements:

The `close` method of `InputStream` does nothing.

Throws:

`IOException` - if an I/O error occurs.

mark

```
public void mark(int readlimit)
```

Marks the current position in this input stream. A subsequent call to the `reset` method repositions this stream at the last marked position so that subsequent reads re-read the same bytes.

The `readlimit` arguments tells this input stream to allow that many bytes to be read before the mark position gets invalidated.

The general contract of `mark` is that, if the method `markSupported` returns `true`, the stream somehow remembers all the bytes read after the call to `mark` and stands ready to supply those same bytes again if and whenever the method `reset` is called. However, the stream is not required to remember any data at all if more than `readlimit` bytes are read from the stream before `reset` is called.

Marking a closed stream should not have any effect on the stream.

Implementation Requirements:

The `mark` method of `InputStream` does nothing.

Parameters:

`readlimit` - the maximum limit of bytes that can be read before the mark position becomes invalid.

See Also:

`reset()`

reset

```
public void reset()
           throws IOException
```

Repositions this stream to the position at the time the `mark` method was last called on this input stream.

The general contract of `reset` is:

- If the method `markSupported` returns `true`, then:
    - If the method `mark` has not been called since the stream was created, or the number of bytes read from the stream since `mark` was last called is larger than the argument to `mark` at that last call, then an `IOException` might be thrown.
    - If such an `IOException` is not thrown, then the stream is reset to a state such that all the bytes read since the most recent call to `mark` (or since the start of the file, if `mark` has not been called) will be resupplied to subsequent callers of the `read` method, followed by any bytes that otherwise would have been the next input data as of the time of the call to `reset`.
- If the method `markSupported` returns `false`, then:
    - The call to `reset` may throw an `IOException`.
    - If an `IOException` is not thrown, then the stream is reset to a fixed state that depends on the particular type of the input stream and how it was created. The bytes that will be supplied to subsequent

callers of the `read` method depend on the particular type of the input stream.

Implementation Requirements:

The method `reset` for class `InputStream` does nothing except throw an `IOException`.

Throws:

`IOException` - if this stream has not been marked or if the mark has been invalidated.

See Also:

`mark(int)`, `IOException`

markSupported

```
public boolean markSupported()
```

Tests if this input stream supports the `mark` and `reset` methods. Whether or not `mark` and `reset` are supported is an invariant property of a particular input stream instance.

Implementation Requirements:

The `markSupported` method of `InputStream` returns `false`.

Returns:

`true` if this stream instance supports the mark and reset methods; `false` otherwise.

See Also:

`mark(int)`, `reset()`

transferTo

```
public long transferTo(OutputStream out)
                throws IOException
```

Reads all bytes from this input stream and writes the bytes to the given output stream in the order that they are read. On return, this input stream will be at end of stream. This method does not close either stream.

This method may block indefinitely reading from the input stream, or writing to the output stream. The behavior for the case where the input and/or output stream is *asynchronously closed*, or the thread interrupted during the transfer, is highly input and output stream specific, and therefore not specified.

If the total number of bytes transferred is greater than Long.MAX_VALUE, then `Long.MAX_VALUE` will be returned.

If an I/O error occurs reading from the input stream or writing to the output stream, then it may do so after some bytes have been read or written. Consequently the input stream may not be at end of stream and one, or both, streams may be in an inconsistent state. It is strongly recommended that both streams be promptly closed if an I/O error occurs.

Parameters:

`out` - the output stream, non-null

Returns:

the number of bytes transferred

Throws:

`IOException` - if an I/O error occurs when reading or writing

`NullPointerException` - if `out` is `null`

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.